

UNIVERSIDAD DE SONORA



**"El saber de mis hijos
hará mi grandeza"**

Desarrollo de Sistemas IV

Act. 7 Proyecto final: API REST

Maestro: Ivan Dostoyewski Meza Ibarra

Integrantes:

**Árevalo Coronado María José
Marquez Martinez Perla Jazmín
Román Ruiz María Celeste**

**Carrera: Ingeniería En Sistemas
De Información**

Fecha: 12/05/2026

Índice

- 1. Introducción**
- 2. Desarrollo**
 - a. Descripción del sistema: Problema, entidades.**
 - b. Modelo de datos: Tablas.**
 - c. Endpoints: Lista de rutas**
 - d. Evidencias: Capturas de pruebas**
- 3. Conclusiones**
- 4. Referencias Bibliográficas**

1. Introducción

En este proyecto se desarrolló una API REST para la administración de una pizzería, utilizando Flask como framework principal en Python, SQLite como gestor de base de datos y una interfaz web en HTML y JavaScript para facilitar la interacción del usuario. El sistema permite realizar operaciones CRUD (crear, consultar, actualizar y eliminar) sobre las entidades del sistema, además de manejar información en formato JSON mediante endpoints HTTP.

Asimismo, se aplicaron conceptos aprendidos en desarrollo de sistemas y de bases de datos, como el uso de tablas relacionadas, validación de datos, manejo de errores y medidas básicas de seguridad para evitar vulnerabilidades como SQL Injection.

El objetivo principal de este proyecto es mostrar el funcionamiento de una API REST funcional conectada a una base de datos, ofreciendo una solución práctica para la gestión digital del catálogo de productos de una pizzería.

2. Desarrollo

a. Descripción del sistema: Problema, entidades.

2.1 Problema

Una pizzería necesita un sistema para gestionar su catálogo de productos. En la actualidad los datos se llevan de forma manual, lo que genera errores y pérdida de información, se requiere una solución que autorice:

- Registrar pizzas con nombre, tamaño, precio y descripción.
- Relacionar ingredientes a cada pizza con su categoría (queso, vegetal, carne y salsa).
- Consultar, crear, editar y eliminar registros de manera sencilla.

2.2 Solución propuesta

Se creó una API REST con Flask para administrar la información de la pizzería. Además, se creó una interfaz web en HTML y JavaScript para que el usuario pueda usar el sistema de forma sencilla desde el navegador.

2.3 Entidades

Entidad	Relación	Atributos
Pizzas	Tabla principal	id(PK), nombre, tamaño, precio, descripción
Ingredientes	N por pizza	id(PK), nombre, tipo, pizza_id(FK)

b. Modelo de Datos: Tablas

Tabla Pizzas

Campo	Tipo	Restricciones
id	INTEGER	PRIMARY KEY, AUTOINCREMENT
nombre	TEXT	NOT NULL
tamaño	TEXT	NOT NULL, CHECK(chica/mediana/grande)
precio	REAL	NOT NULL
descripción	TEXT	Opcional

Tabla Ingredientes

Campo	Tipo	Restricciones
id	INTEGER	PRIMARY KEY, AUTOINCREMENT
nombre	TEXT	NOT NULL
tipo	TEXT	NOT NULL, CHECK(queso/vegetal/carne/salsa)
pizza_id	INTEGER	NOT NULL, FOREIGN KEY - pizzas(id) ON DELETE CASCADE

c. Endpoints: Lista de Rutas

Detalle de Endpoints:

1er Tabla - Sección PIZZAS

- Obtener los registros en la tabla de pizzas.
GET /pizzas

HTTP PizzeriaMPC > GET

GET http://127.0.0.1:5000/pizzas Send

Docs Params Authorization Headers 7 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body Cookies Headers 5 Test Results 200 OK • 72 ms • 355 B • Save Response

{ } JSON Preview Visualize

```
1 [
2   {
3     "descripcion": "Salsa de tomate, queso mozzarella y trozos de pepperoni.",
4     "id": 1,
5     "nombre": "Pizza de Pepperoni",
6     "precio": 100.0,
7     "tamaño": "mediana"
8   }
9 ]
10
```

- Añadir/Crear una pizza.
POST /pizzas

HTTP PizzeriaMPC > post

POST http://127.0.0.1:5000/pizzas Send

Docs Params Authorization Headers 9 Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1 {
2   "nombre": "Margarita",
3   "tamaño": "mediana",
4   "precio": 120.50,
5   "descripcion": "Clásica italiana con albahaca"
6 }
```

Body Cookies Headers 5 Test Results 201 CREATED • 28 ms • 213 B • Save Response

{ } JSON Preview Visualize

```
1 {
2   "id": 2,
3   "mensaje": "Pizza creada"
4 }
5
```

- Obtener información de las pizzas por ID.
GET /pizzas/<id>

HTTP PizzeriaMPC > **getID** Save Share

GET http://127.0.0.1:5000/pizzas/1 Send

Docs Params Authorization Headers 7 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body Cookies Headers 5 Test Results 200 OK • 11 ms • 668 B Save Response

{ } JSON Preview Visualize

```
1 {
2   "descripcion": "Salsa de tomate, queso mozzarella y trozos de pepperoni.",
3   "id": 1,
4   "ingredientes": [
5     {
6       "id": 1,
7       "nombre": "Mozzarella",
8       "pizza_id": 1,
9       "tipo": "queso"
10    },
11    {
12      "id": 2,
13      "nombre": "Pepperoni",
14      "pizza_id": 1,
15      "tipo": "carne"
16    },
17    {
18      "id": 3,
19      "nombre": "Salsa de tomate",
20      "pizza_id": 1,
21      "tipo": "salsa"
22    }
23  ],
24  "nombre": "Pizza de Pepperoni",
25  "precio": 100.0,
26  "tamaño": "mediana"
27 }
```

- Actualizar información de una pizza
PUT /pizzas/<id>

HTTP PizzeriaMPC > **PUT** Save Share

PUT http://127.0.0.1:5000/pizzas/1 Send

Docs Params Authorization Headers 9 Body Scripts Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON Schema Beautify

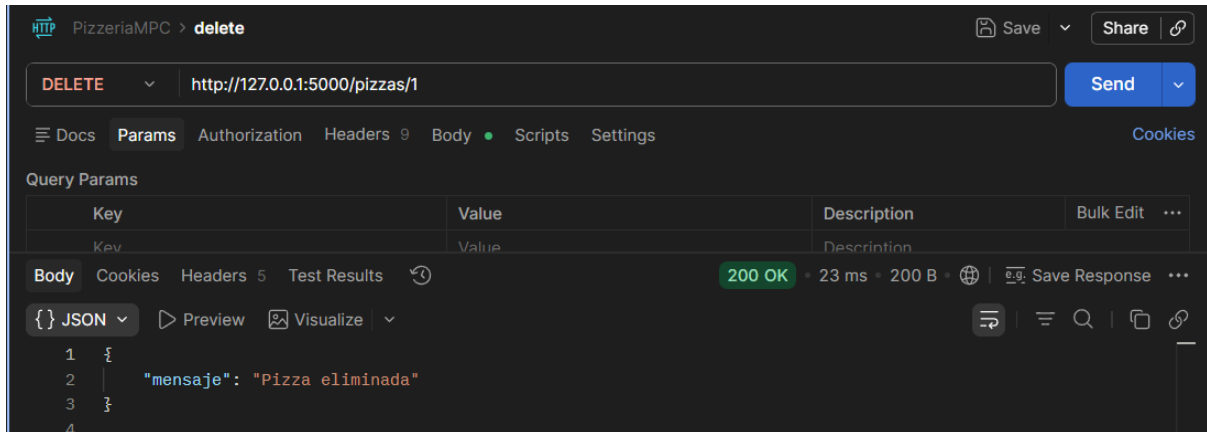
```
1 {
2   "precio": 150.00,
3   "descripcion": "Versión premium"
4 }
```

Body Cookies Headers 5 Test Results 200 OK • 20 ms • 202 B Save Response

{ } JSON Preview Visualize

```
1 {
2   "mensaje": "Pizza actualizada"
3 }
4
```

- Eliminar una pizza
DELETE /pizzas/<id>



HTTP PizzeriaMPC > delete

DELETE http://127.0.0.1:5000/pizzas/1

Send

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body Cookies Headers 5 Test Results

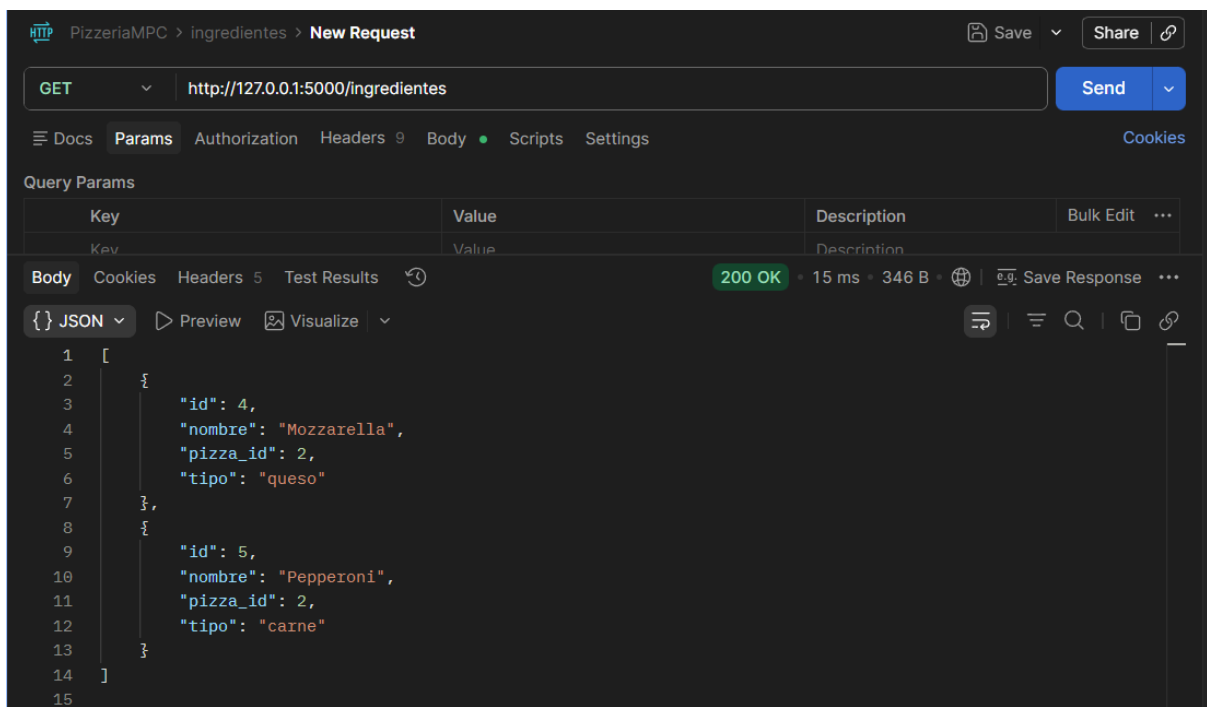
200 OK • 23 ms • 200 B • Save Response

{ } JSON Preview Visualize

```
1 {
2   "mensaje": "Pizza eliminada"
3 }
4
```

2da Tabla - Sección INGREDIENTES

- Obtener los ingredientes registrados.
GET /ingredientes



HTTP PizzeriaMPC > ingredientes > New Request

GET http://127.0.0.1:5000/ingredientes

Send

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body Cookies Headers 5 Test Results

200 OK • 15 ms • 346 B • Save Response

{ } JSON Preview Visualize

```
1 [
2   {
3     "id": 4,
4     "nombre": "Mozzarella",
5     "pizza_id": 2,
6     "tipo": "queso"
7   },
8   {
9     "id": 5,
10    "nombre": "Pepperoni",
11    "pizza_id": 2,
12    "tipo": "carne"
13  }
14 ]
15
```

- Agregar un ingrediente
POST /ingredientes

The screenshot shows a REST client interface for a project named "PizzeriaMPC". The selected endpoint is "ingredientes" with a "POST" method. The URL is "http://127.0.0.1:5000/ingredientes". The request body is a JSON object:

```
{  "nombre": "Menonita",  "tipo": "queso",  "pizza_id": 2}
```

. The response status is "201 CREATED" with a time of 21 ms and a size of 219 B. The response body is a JSON object:

```
{  "id": 6,  "mensaje": "Ingrediente creado"}
```

. The interface includes tabs for Docs, Params, Authorization, Headers, Body, Scripts, and Settings. The Body tab is active, showing the request and response JSON.

- Obtener un ingrediente por ID
GET /ingredientes/<id>

The screenshot shows a REST client interface for a project named "PizzeriaMPC". The selected endpoint is "getID" with a "GET" method. The URL is "http://127.0.0.1:5000/ingredientes/4". The response status is "200 OK" with a time of 10 ms and a size of 241 B. The response body is a JSON object:

```
{  "id": 4,  "nombre": "Mozzarella",  "pizza_id": 2,  "tipo": "queso"}
```

. The interface includes tabs for Docs, Params, Authorization, Headers, Body, Scripts, and Settings. The Body tab is active, showing the response JSON. Above the response, there is a "Query Params" table.

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

- Actualizar información de un ingrediente
PUT /ingredientes/<id>

The screenshot shows a REST client interface for a PizzeriaMPC API. The request is a PUT to `http://127.0.0.1:5000/ingredientes/4`. The request body is raw JSON: `{ "nombre": "Mozzarella fresca", "tipo": "queso" }`. The response is a 200 OK with a JSON body: `{ "mensaje": "Ingrediente actualizado" }`.

```
1 {
2   "nombre": "Mozzarella fresca",
3   "tipo": "queso"
4 }
```

```
1 {
2   "mensaje": "Ingrediente actualizado"
3 }
4
```

- Eliminar un ingrediente
DELETE /ingredientes/<id>

The screenshot shows a REST client interface for a PizzeriaMPC API. The request is a DELETE to `http://127.0.0.1:5000/ingredientes/4`. The response is a 200 OK with a JSON body: `{ "mensaje": "Ingrediente eliminado" }`.

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

```
1 {
2   "mensaje": "Ingrediente eliminado"
3 }
4
```

- POST con datos faltantes
POST /pizzas

The screenshot shows a REST client interface for a PizzeriaMPC API. The request is a POST to `http://127.0.0.1:5000/pizzas` with a JSON body: `{ "nombre": "Suprema" }`. The response is a `400 BAD REQUEST` with a message: `"error": "Falta campo: tamaño"`.

```
1 {
2   "nombre": "Suprema"
3 }
```

400 BAD REQUEST • 16 ms • 216 B • Save Response

```
1 {
2   "error": "Falta campo: tamaño"
3 }
```

- POST con tamaño invalido
POST /pizzas

The screenshot shows a REST client interface for a PizzeriaMPC API. The request is a POST to `http://127.0.0.1:5000/pizzas` with a JSON body: `{ "nombre": "Suprema", "tamaño": "jumbo", "precio": 99 }`. The response is a `400 BAD REQUEST` with a message: `"error": "tamaño debe ser chica, mediana o grande"`.

```
1 {
2   "nombre": "Suprema", "tamaño": "jumbo", "precio": 99
3 }
```

400 BAD REQUEST • 53 ms • 236 B • Save Response

```
1 {
2   "error": "tamaño debe ser chica, mediana o grande"
3 }
```

- GET con ID inexistente
GET /pizzas/999

The screenshot shows a REST client interface for a PizzeriaMPC API. The request is a GET to `http://127.0.0.1:5000/pizzas/999`. The response is a `404 NOT FOUND` with a message: `"error": "Pizza no encontrada"`.

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

404 NOT FOUND • 23 ms • 209 B • Save Response

```
1 {
2   "error": "Pizza no encontrada"
3 }
```

d. Evidencias: Capturas de pruebas

Pruebas de la Interfaz HTML

Pantalla Inicial sin registros:

The screenshot shows the initial interface of the Pizzería MPC application. At the top, there is a red header with the logo and the text "Pizzería MPC" and "API REST - Gestión de pizzas e ingredientes". Below the header, there are two tabs: "Pizzas" and "Ingredientes". The "Pizzas" tab is active, showing a summary of "0 PIZZAS REGISTRADAS" and "0 INGREDIENTES REGISTRADOS". Below this, there is a section titled "Gestión de Pizzas" with a form to "Agregar Pizza". The form has three input fields: "Nombre" (with the example "Ej: Margarita"), "Tamaño" (a dropdown menu with "Selecciona" selected), and "Precio (\$)" (with the value "99.00"). There is also a "Descripción" field with the placeholder "Ingredientes destacados...". At the bottom of the form are two buttons: "Guardar" (red) and "Cancelar" (gray). Below the form, there is a message: "¡Aún no tienes pizzas!".

Creación de una pizza:

The screenshot shows the same interface as the previous one, but with the "Agregar Pizza" form filled out. The "Nombre" field now contains "Pizza de Queso", the "Tamaño" dropdown menu is set to "Mediana", and the "Precio (\$)" field shows "100". The "Descripción" field now contains "Salsa de Tomate y Queso". The "Guardar" button is still red, and the "Cancelar" button is still gray. The message "¡Aún no tienes pizzas!" is still present at the bottom.

1

PIZZAS REGISTRADAS

0

INGREDIENTES REGISTRADOS

Pizza creada

Gestión de Pizzas

Agregar Pizza

Nombre

Tamaño

Precio (\$)

Ej: Margarita

— Selecciona —

99.00

Descripción

Ingredientes destacados...

Guardar

Cancelar

Pizza de Queso

Mediana

\$100.00

Salsa de Tomate y Queso

Editar

Eliminar

Listado de Pizzas registradas

3

PIZZAS REGISTRADAS

0

INGREDIENTES REGISTRADOS

Gestión de Pizzas

Agregar Pizza

Nombre

Tamaño

Precio (\$)

Ej: Margarita

— Selecciona —

99.00

Descripción

Ingredientes destacados...

Guardar

Cancelar

Pizza de Queso

Mediana

\$100.00

Salsa de Tomate y Queso

Editar

Eliminar

Pizza de Pepperoni

Grande

\$120.00

Salsa de tomate, queso y pepperoni

Editar

Eliminar

Pizza Mexicana

Chica

\$99.00

Salsa de tomate, queso, frijoles, chorizo y jalapeños.

Editar

Eliminar

Edición de una Pizza (Mexicana)

3PIZZAS REGISTRADAS

0INGREDIENTES REGISTRADOS

Pizza actualizada

Gestión de Pizzas

Agregar Pizza

Nombre

Ej: Margarita

Tamaño

— Selecciona —

Precio (\$)

99.00

Descripción

Ingredientes destacados...

Guardar

Cancelar

Pizza de Queso

Mediana\$100.00

Salsa de Tomate y Queso

Editar

Eliminar

Pizza de Pepperoni

Grande\$120.00

Salsa de tomate, queso y pepperoni

Editar

Eliminar

Pizza Mexicana

Chica\$99.00

Salsa de tomate, queso, frijoles y chorizo.

Editar

Eliminar

Visualización de ingredientes

3PIZZAS REGISTRADAS

4INGREDIENTES REGISTRADOS

Gestión de Ingredientes

Agregar Ingrediente

Nombre

Ej: Mozzarella

Tipo

— Selecciona —

Pizza

4 — Pizza de Queso

Guardar

Cancelar

Salsa de Tomate

Salsa

Pizza: Pizza de Queso

Editar

Eliminar

Mozzarella

Queso

Pizza: Pizza de Queso

Editar

Eliminar

Pepperoni

Carne

Pizza: Pizza de Pepperoni

Editar

Eliminar

Jalapeño

Vegetal

Pizza: Pizza Mexicana

Editar

Eliminar

Validaciones

3
PIZZAS REGISTRADAS

4
INGREDIENTES REGISTRADOS

Selecciona un tamaño.

Gestión de Pizzas

Agregar Pizza

Nombre	Tamaño	Precio (\$)
Pizza Italiana	— Selecciona —	130
Descripción Salsa de Tomate, queso, albahaca, aceite de oliva.		
Guardar Cancelar		

3. Conclusiones

Al inicio del proyecto pensábamos que hacer una API REST sería simplemente "escribir funciones que devuelven datos", pero conforme avanzamos nos dimos cuenta de que detrás hay mucho más: elegir el método HTTP correcto para cada acción, devolver el código de estado adecuado y proteger la base de datos para que no se vea afectada por entradas maliciosas. Entender la diferencia entre POST y PUT nos costó al principio, pero una vez aclarada, diseñar los 10 endpoints se volvió bastante natural.

Lo que más nos sorprendió fue ver en acción la regla ON DELETE CASCADE: al eliminar una pizza, sus ingredientes desaparecían automáticamente. Eso nos hizo comprender la verdadera utilidad de las claves foráneas que tantas veces vimos como teoría. Además, conectar la interfaz con fetch a nuestra propia API fue la parte más satisfactoria, porque pudimos observar el ciclo completo: formulario → JSON → backend → base de datos → respuesta renderizada en pantalla sin recargar. Nos llevamos dos aprendizajes que seguramente nos servirán adelante: nunca confiar en los datos que llegan del frontend, y que un buen diseño de base de datos ahorra muchos problemas en el camino.

4. Referencias Bibliográficas

sqlite3 — DB-API 2.0 interface for SQLite databases. (s. f.). Python Documentation.

<https://docs.python.org/3/library/sqlite3.html>

SQL Injection | OWASP Foundation. (s. f.).

https://owasp.org/www-community/attacks/SQL_Injection

Uso de Fetch - API web | MDN. (2025, 27 marzo).

https://developer.mozilla.org/es/docs/Web/API/Fetch_API/Using_Fetch

Parzibyte. (2020, 10 noviembre). API REST con Python, Flask y SQLite3. *Parzibyte*.

<https://parzibyte.me/blog/posts/api-rest-python-flask-sqlite3/>

Dimas. (2021, 23 septiembre). 🍹 *FLASK - REST API Simple con Python [con SQLite y*

SQLAlchemy] [Video]. YouTube. <https://www.youtube.com/watch?v=4fcStD9kLcI>

NDC. (2025, 12 mayo). *Python Backend #3: Connect Flask to a Database (SQLite CRUD*

Tutorial) [Video]. YouTube. <https://www.youtube.com/watch?v=UqH77WhRLH0>

Materiales de clase — Interfaz de Usuario (Unidad 9). 2026.